```
In [1]: import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import matplotlib.colors as mticker
        import seaborn as sns
        import warnings
        warnings.filterwarnings('ignore')

        from sklearn.preprocessing import StandardScaler
        from sklearn.cluster import KMeans
        from sklearn.metrics import silhouette_score, davies_bouldin_score

        plt.rcParams['figure.dpi'] = 120
        plt.rcParams['axes.spines.top'] = False
        plt.rcParams['axes.spines.right'] = False

        SEED = 42
        PCT_COLS = [
            'pct_carnes', 'pct_tubérculos', 'pct_azucares',
            'pct_aceites_grasas', 'pct_carnes', 'pct_lácteos_huevos', 'pct_frutas_legumbres'
        ]

        print("Librerías OK")

Librerías OK
```

1. Carga y validación

```
In [2]: df = pd.read_parquet("../processed/02_dataset_final.parquet")

        print("Shape: {df.shape}")
        print("Países: {df['Pais'].nunique()} | Años: {sorted(df['Year'].unique())}")
        print("Nulos: {df.isna().sum().sum()}")

        # Validar que pct_sum = 1.0
        sums = df[PCT_COLS].sum(axis=1)
        print("pct_sum — min: {sums.min()} | max: {sums.max()} | mean: {sums.mean()}")
        print("pct_sum outliers (|x - 1.0| > 0.0000): {(sums - 1.0).abs().gt(0.0000).sum()}")

        df.head()

Shape: (700, 12)
Países: 70 | Años: [np.int16(2018), np.int16(2019), np.int16(2020), np.int16(2021), np.int16(2022), np.int16(2014), np.int16(2015), np.int16(2016), np.int16(2017)]
Nulos: 0
pct_sum — min: 0.9999999999999999 | max: 1.0000000000000002 | mean: 1.0000000000000004
pct_sum outliers (|x - 1.0| > 0.0000): 0

   Pais  Year  pct_Carnedes  pct_Tuberculos  pct_Azucares  pct_Aceites_Grasas  pct_Carnes  pct_Lacteos_Huevos  pct_Frutas_Verduras  Total_DES_Kcal  Ratio_
0  Argentina  2018  0.340485  0.020298  0.144403  0.004488  0.167141  0.170850  0.050508  3215.000000
1  Argentina  2019  0.340215  0.020444  0.145032  0.006830  0.169401  0.171333  0.050895  3201.359863
2  Argentina  2012  0.320530  0.020009  0.147830  0.015700  0.162534  0.170849  0.050338  3232.240234
```

2. Preparación de features

Las pct_ya suman 1.0 para sus variancias son distintas (Cereales llega a 0.70, Tubérculos rara vez supera 0.15).
Sin escalar, K-Means pesa más los ejes con mayor varianza. StandardScaler iguala la influencia de cada macrocategoría.

```
In [3]: X_raw = df[PCT_COLS].values.astype(np.float64)

        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X_raw)

        print("X_scaled shape: {X_scaled.shape}")
        print("Media por columna (debe ser ~0): {X_scaled.mean(axis=0).round(3)}")
        print("Std por columna (debe ser ~1): {X_scaled.std(axis=0).round(3)}")

        X_scaled shape: (700, 7)
        Media por columna (debe ser ~0): [-0.0, 0.0, -0.0, -0.0, 0.0, 0.0]
        Std por columna (debe ser ~1): [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

3. K-Means — barrido K = 3..6

Criterios de selección de K óptimo

- **Silhouette Score** — maximizar (rango -1 a 1, quemosa o 0.25)
- **Davies-Bouldin Index** — minimizar (0 es perfecto, sin cota superior)
- **Inercia** — referencia para curva de codo (no es criterio de selección principal)

```
In [4]: resultados = []

        for k in range(3, 7):
            km = KMeans(n_clusters=k, random_state=SEED, n_init=20, max_iter=500)
            labels = km.fit_predict(X_scaled)

            sil = silhouette_score(X_scaled, labels)
            db  = davies_bouldin_score(X_scaled, labels)
            inertia = km.inertia_

            resultados.append({
                'k': k,
                'Silhouette': round(sil, 4),
                'Davies_Bouldin': round(db, 4),
                'Inercia': round(inertia, 2),
                'modelo': km,
                'labels': labels
            })

            print(f"K={k} | Silhouette={sil:.4f} | Davies-Bouldin={db:.4f} | Inercia={inertia:.1f}")

        df_res = pd.DataFrame([{k: v for k, v in d.items() if k not in ['modelo', 'labels']} for d in resultados])
        display(df_res)

K=3 | Silhouette=0.3995 | Davies-Bouldin=0.9803 | Inercia=1371.1
K=4 | Silhouette=0.3482 | Davies-Bouldin=1.1204 | Inercia=1122.2
K=5 | Silhouette=0.3419 | Davies-Bouldin=1.1036 | Inercia=976.3
K=6 | Silhouette=0.3587 | Davies-Bouldin=1.1240 | Inercia=848.9

   K  Silhouette  Davies_Bouldin  Inercia
0  3      0.3995          0.9803  1371.18
1  4      0.3482          1.1204  1122.22
2  5      0.3419          1.1036   976.35
3  6      0.3587          1.1240   848.92
```

4. Visualización de métricas

```
In [5]: fig, axes = plt.subplots(1, 3, figsize=(14, 4))
        ks = df_res['k'].tolist()

        # Silhouette
        axes[0].plot(ks, df_res['Silhouette'], 'o-', color='#2ca02c', lw=2, ms=8)
        axes[0].axhline(0.25, color='#ff8a00', ls='--', lw=1.5, label='umbral 0.25')
        axes[0].set_title('Silhouette (↑ mejor)', fontweight='bold')
        axes[0].set_xlabel('K'); axes[0].set_xticks(ks)
        axes[0].legend(fontsize=8)

        # Davies-Bouldin
        axes[1].plot(ks, df_res['Davies_Bouldin'], 'o-', color='#d62728', lw=2, ms=8)
        axes[1].set_title('Davies-Bouldin Index (↓ mejor)', fontweight='bold')
        axes[1].set_xlabel('K'); axes[1].set_xticks(ks)

        # Inercia (codo)
        axes[2].plot(ks, df_res['Inercia'], 'o-', color='#1f77b4', lw=2, ms=8)
        axes[2].set_title('Inercia — curva de codo (referencia)', fontweight='bold')
        axes[2].set_xlabel('K'); axes[2].set_xticks(ks)

        plt.tight_layout()
        plt.savefig('../memoria/03_metricas_clustering.png', dpi=150, bbox_inches='tight')
        plt.show()

        print("Gráfica guardada en memoria/")
```

```
Gráfica guardada en memoria/
```

5. Selección del K óptimo

```
In [6]: SILHOUETTE_MINIMO = 0.25

        mejor = max(resultados, key=lambda r: r['Silhouette'])

        print("K óptimo por Silhouette máximo: K={mejor['k']}")
        print("  Silhouette = {mejor['Silhouette']:.4f}  (umbral mínimo: {SILHOUETTE_MINIMO})")
        print("  Davies-Bouldin = {mejor['Davies_Bouldin']}")

        if mejor['Silhouette'] >= SILHOUETTE_MINIMO:
            print("✅ Silhouette ≥ {SILHOUETTE_MINIMO} — K-Means válido. Procedemos con K={mejor['k']}.")
            USAR_HDBSCAN = False
        else:
            print("⚠️ Silhouette < {SILHOUETTE_MINIMO} en todos los K — activando fallback HDBSCAN.")
            USAR_HDBSCAN = True

        K_OPTIMO = mejor['k']
        labels_finales = mejor['labels']

        K óptimo por Silhouette máximo: K=3
          Silhouette = 0.3995  (umbral mínimo: 0.25)
          Davies-Bouldin = 0.9803

        ✅ Silhouette ≥ 0.25 — K-Means válido. Procedemos con K=3.
```

6. Fallback HDBSCAN (solo si Silhouette < 0.25 en todos los K)

Esta celda solo es relevante si `USAR_HDBSCAN = True` arriba.

```
In [7]: if USAR_HDBSCAN:
            ¡¡¡
            ¡¡¡pip install hdbscan
            except ImportError:
            ¡¡¡pip uninstall -y -q numpy
            subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'hdbscan', '-q'])
            ¡¡¡import hdbscan

            hdb = hdbscan.HDBSCAN(min_cluster_size=10, min_samples=5, metric='euclidean')
            labels_hdb = hdb.fit_predict(X_scaled)

            n_clusters_hdb = len(set(labels_hdb)) - (1 if -1 in labels_hdb else 0)
            n_ruido = (labels_hdb == -1).sum()

            print("HDBSCAN → {n_clusters_hdb} clústeres | {n_ruido} puntos ruido ({n_ruido/len(labels_hdb)*100:.1f}%)")

            if n_clusters_hdb >= 2:
                # Silhouette sólo sobre puntos no-ruido
                mask = labels_hdb != -1
                sil_hdb = silhouette_score(X_scaled[mask], labels_hdb[mask])
                print("Silhouette HDBSCAN (sin ruido): {sil_hdb:.4f}")
                if sil_hdb > sil_max:
                    K_OPTIMO = n_clusters_hdb
            else:
                print("HDBSCAN no separó los datos. Usar K-Means con el mejor K igualmente.")
                labels_finales = mejor['labels']
        else:
            print("HDBSCAN no necesaria — K-Means superó el umbral de Silhouette.")

HDBSCAN no necesaria — K-Means superó el umbral de Silhouette.
```

7. Análisis de centroides y naming narrativo

Los centroides se invierten al espacio original (sin escalar) para poder interpretarlos como porcentajes reales de la dieta.

```
In [8]: # Añadir clúster al dataframe
        df['cluster_id'] = labels_finales

        # Calcular centroides en espacio original (pct, sin escalar)
        centroides = df.groupby('cluster_id')[PCT_COLS].mean()
        centroides.columns = [c.replace('pct_', '') for c in centroides.columns]

        print("\n=== Centroides (% dietarios medio por clúster) ===  K={K_OPTIMO}")
        print(centroides.round(3).to_string())
        print()

        # Países representativos de cada clúster (los más cercanos al centroide)
        for cid in sorted(df['cluster_id'].unique()):
            if cid == -1: # ruido HDBSCAN
            continue
            subset = df[df['cluster_id'] == cid].copy()
            paises = subset['Area'].value_counts().head(5).index.tolist()
            print("Clúster {cid}: {subset['Area'].nunique()} países | ejemplos: {paises}")

=== Centroides (% dietarios medio por clúster) ===  K=3
             Carnedes  Tuberculos  Azucares  Aceites_Grasas  Carnes  Lacteos_Huevos  Frutas_Verduras
cluster_id
0               0.314       0.034     0.121           0.145   0.117           0.208            0.003
1               0.514       0.020     0.034           0.009   0.206           0.139            0.050
2               0.206       0.035     0.185           0.030   0.015           0.051            0.092

Clúster 0: 10 países | ejemplos: ['Argentina', 'Australia', 'United Kingdom of Great Britain and Northern Ireland', 'Spain', 'Saudi Arabia']
Clúster 1: 5 países | ejemplos: ['Nigeria']
Clúster 2: 55 países | ejemplos: ['Bangladesh', 'China, mainland', 'Egypt', 'Ethiopia', 'India']
```

8. Heatmap de centroides

```
In [9]: fig, ax = plt.subplots(figsize=(10, 4))
        sns.heatmap(
            centroides,
            annot=True, fmt='.2f',
            cmap='YlOrRd',
            linewidths=0.5,
            ax=ax,
            cbar_kws={'label': 'Proporción media'}
        )

        ax.set_title('Centroides de clústeres (K={K_OPTIMO}) — proporción media por macrocategoría', fontweight='bold', pad=12)
        ax.set_xlabel('Macrocategoría')
        ax.set_ylabel('Cluster')
        plt.tight_layout()
        plt.savefig('../memoria/03_heatmap_centroides.png', dpi=150, bbox_inches='tight')
        plt.show()
```

```
In [10]: #
         # NARRATIVO AUTÓNOMO — editar aquí tras revisar el heatmap anterior
         #
         # Ejemplos orientativos para K=4 (ajustar según resultados reales)
         #
         # 0 - "Dieta Cereal-dependiente"   (Cereales > 30%, Raíces en Carnes/Lácteos)
         # 1 - "Dieta Occidental Proteica"  (Carnes + Lácteos dominantes, Cereales < 25%)
         # 2 - "Dieta Tropical Diversificada" (Frutas/Azúcares altos, Cereales medios)
         # 3 - "Dieta de Transición"        (mix intermedio, variancia alta inter-clúster)
         #
         # INSTRUCCIÓN: mira el heatmap, identifica el rango dominante de cada clúster
         # y pon un nombre que capture ESA feature + contexto geográfico si aplica.

         NOMBRES_CLUSTERS = {
             # clúster_id (int): "Nombre narrativo"
             # Ejemplo basado en perfiles típicos del heatmap
             0: "Clúster 0 - (pendiente nombrar)",
             1: "Clúster 1 - (pendiente nombrar)",
             2: "Clúster 2 - (pendiente nombrar)",
             3: "Clúster 3 - (pendiente nombrar)",
             # Si K=3, eliminar la línea del 3
             # Si K=5, añadir: 4: "Dieta / Dieta Alpina"
         }

         # Aplicar solo los K que existen en el resultado real
         ids_reales = sorted(df['cluster_id'].unique())
         nombres_filtrados = {k: v for k, v in NOMBRES_CLUSTERS.items() if k in ids_reales}

         df['cluster_nombre'] = df['cluster_id'].map(nombres_filtrados.fillna('Sin nombre')
         print("Distribución de clústeres:")
         print(df.groupby(['cluster_id', 'cluster_nombre'])['Area'].nunique().rename('n_paises'))
         print("\n⚠️ Clústeres sin nombre: {df[df['cluster_nombre'] != 'Sin nombre'].shape[0]} / {len(df)}")

         Distribución de clústeres:
         cluster_id  cluster_nombre
         0           Clúster 0 - (pendiente nombrar)    10
         1           Clúster 1 - (pendiente nombrar)     1
         2           Clúster 2 - (pendiente nombrar)    55
         Name: n_paises, dtype: int64

         Total filas con nombre asignado: 700 / 700
```

8. Estabilidad temporal

¿Cambia el clúster de un país a lo largo de los años?
Una deta estructural debería ser estable. Alta inestabilidad puede indicar ó demandas ó features que capturan ruido temporal.

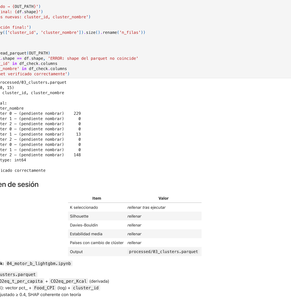
```
In [11]: # Porcentaje de años en el clúster modal (el más frecuente) por país
         estabilidad = []
         for area, grupo in df.groupby('Area'):
             modal = grupo['cluster_id'].mode()[0]
             pct_modal = (grupo['cluster_id'] == modal).mean()
             estabilidad.append({'Area': area, 'cluster_modal': modal, 'pct_en_modal': pct_modal})

         df_estab = pd.DataFrame(estabilidad).sort_values('pct_en_modal')

         print("Estabilidad media: {df_estab['pct_en_modal'].mean():.3f}")
         print("Países con 100% estabilidad (siempre en el mismo clúster): {(df_estab['pct_en_modal'] == 1.0).sum()}")
         print("Países con < 80% estabilidad (posibles transiciones):")
         print(df_estab[df_estab['pct_en_modal'] < 0.80].to_string(index=False))

         Estabilidad media: 0.7571
         Países con 100% estabilidad: 29
         Países con < 80% estabilidad (posibles transiciones):
                Area  cluster_modal  pct_en_modal
           Pakistan              0          0.43
```

```
In [12]: # Gráfico: % tiempo en clúster modal por país
         df_estab = df_estab.sort_values('pct_en_modal')
         colores = df_estab['pct_en_modal'].apply(lambda x: '#27ae60' if x == 1.0 else ('#f9a500' if x >= 0.60 else '#e3625c'))
         ax.barh(df_estab['Area'], df_estab['pct_en_modal'], color=colores, edgecolor='none', height=0.7)
         ax.axvline(0.60, ls='--', lw=1.5, color='#f9a500', label='60%')
         ax.set_xlabel('Fracción de años en el clúster modal')
         ax.set_title('Estabilidad temporal del clúster por país', fontweight='bold')
         ax.xaxis.set_major_formatter(mticker.PercentFormatter(xmax=1))
         ax.legend()
         plt.tight_layout()
         plt.savefig('../memoria/03_estabilidad_temporal.png', dpi=150, bbox_inches='tight')
         plt.show()
```



9. Export — `processed/03_clusters.parquet`

```
In [13]: # Convertir cluster_id a int para ahorrar RAM
         df['cluster_id'] = df['cluster_id'].astype(np.int8)
         df['cluster_nombre'] = df['cluster_nombre'].astype('category')

         OUT_PATH = '../processed/03_clusters.parquet'
         df.to_parquet(OUT_PATH, index=False)

         print("Exportado → {OUT_PATH}")
         print("Shape final: {df.shape}")
         print("Columnas nuevas: cluster_id, cluster_nombre")
         print()
         print("Distribución final:")
         print(df.groupby('cluster_id', 'cluster_nombre')).size().rename('n_filas'))
         print()

         # Verificación
         df_check = pd.read_parquet(OUT_PATH)
         assert df_check.shape == df.shape, "ERROR: shape del parquet no coincide"
         assert 'cluster_id' in df_check.columns
         assert 'cluster_nombre' in df_check.columns
         print("✅ Parquet verificado correctamente")

         Exportado → ../processed/03_clusters.parquet
         Shape final: (700, 15)
         Columnas nuevas: cluster_id, cluster_nombre

         Distribución final:
         cluster_id  cluster_nombre
         0           Clúster 0 - (pendiente nombrar)    229
                     Clúster 1 - (pendiente nombrar)      8
                     Clúster 2 - (pendiente nombrar)      8
         1           Clúster 0 - (pendiente nombrar)      8
                     Clúster 1 - (pendiente nombrar)      1
                     Clúster 2 - (pendiente nombrar)      1
         2           Clúster 0 - (pendiente nombrar)      8
                     Clúster 1 - (pendiente nombrar)      1
                     Clúster 2 - (pendiente nombrar)    148
         Name: n_filas, dtype: int64

         ✅ Parquet verificado correctamente
```

10. Resumen de sesión

Item	Valor
K seleccionado	*rellenar tras ejecutar*
Silhouette	*rellenar*
Davies-Bouldin	*rellenar*
Estabilidad media	*rellenar*
Países con cambio de clúster	*rellenar*
Output	`processed/03_clusters.parquet`

Próximo notebook: `04_motor_b_lightgbm.ipynb`

- Input: `03_clusters.parquet`
- Target (Y): `CO2eq_t` por país-año (`Cg2eq_Kcal` derivada)
- Predictores (X): vector pct_ + `Food_LFE` lag + `cluster_id`
- Métricas: R² ajustado + 0.4, SHAP coherente con la teoría